

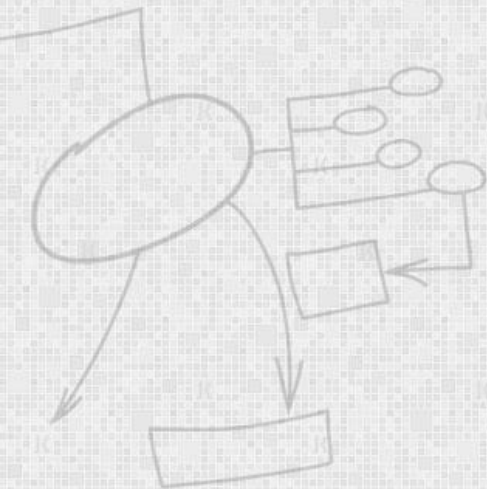
EC-360

SOFTWARE ENGINEERING

Lecture 7

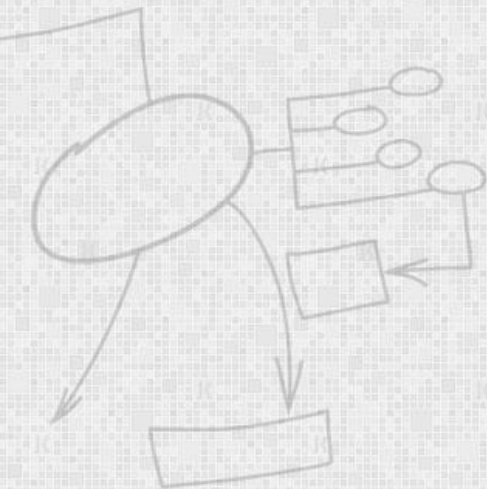
Behavioral Design Patterns

By
Dr Wasi Haider Butt



Agenda

- Observer Pattern
- Memento Pattern



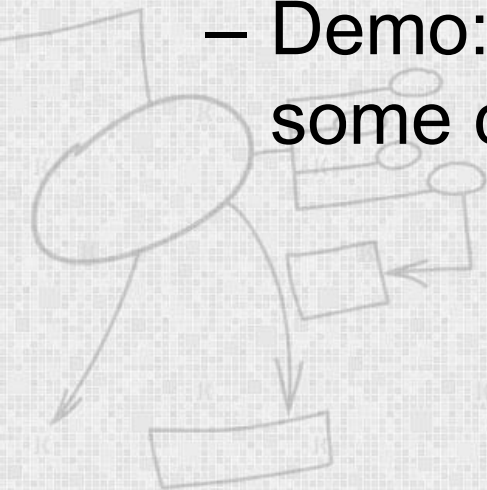
The Observer Design Pattern

- Name: Observer
- Problem: Need to notify a changing number of objects that something has changed
- Solution: Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically



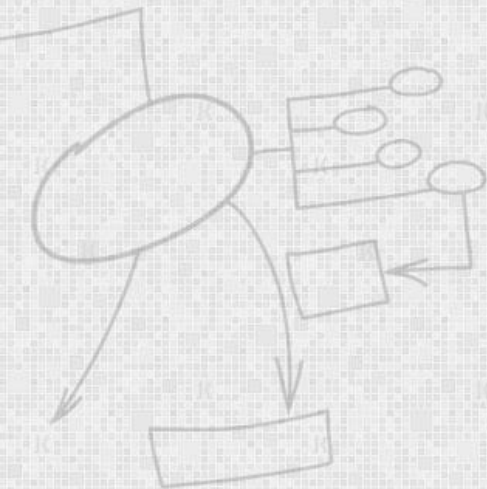
Examples

- Send a newspaper to all who subscribe
 - People add and drop subscriptions, when a new version comes out, it goes to all currently subscribed
- Spreadsheet
 - Demo: Draw two charts—two views—with some changing numbers—the model

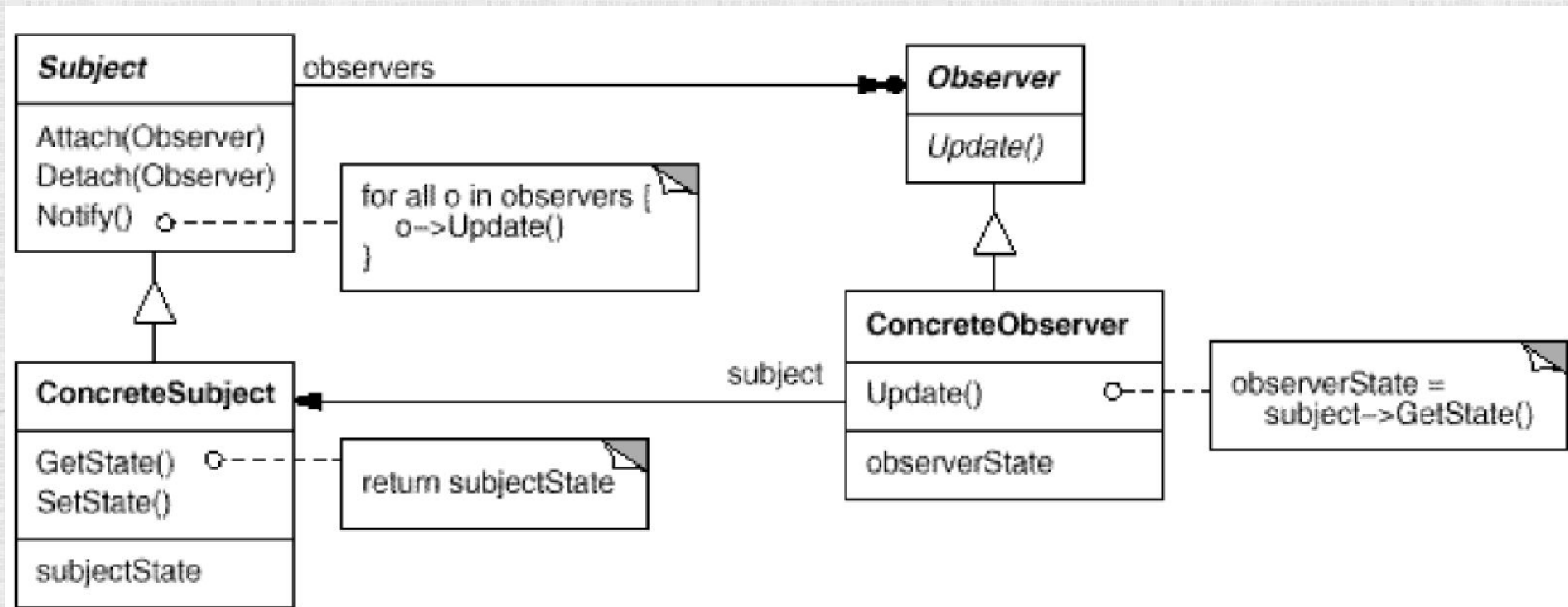


Examples

- File Explorer (or Finders) are registered observers (the view) of the file system (the model).
- Demo: Open several finders to view file system and delete a file

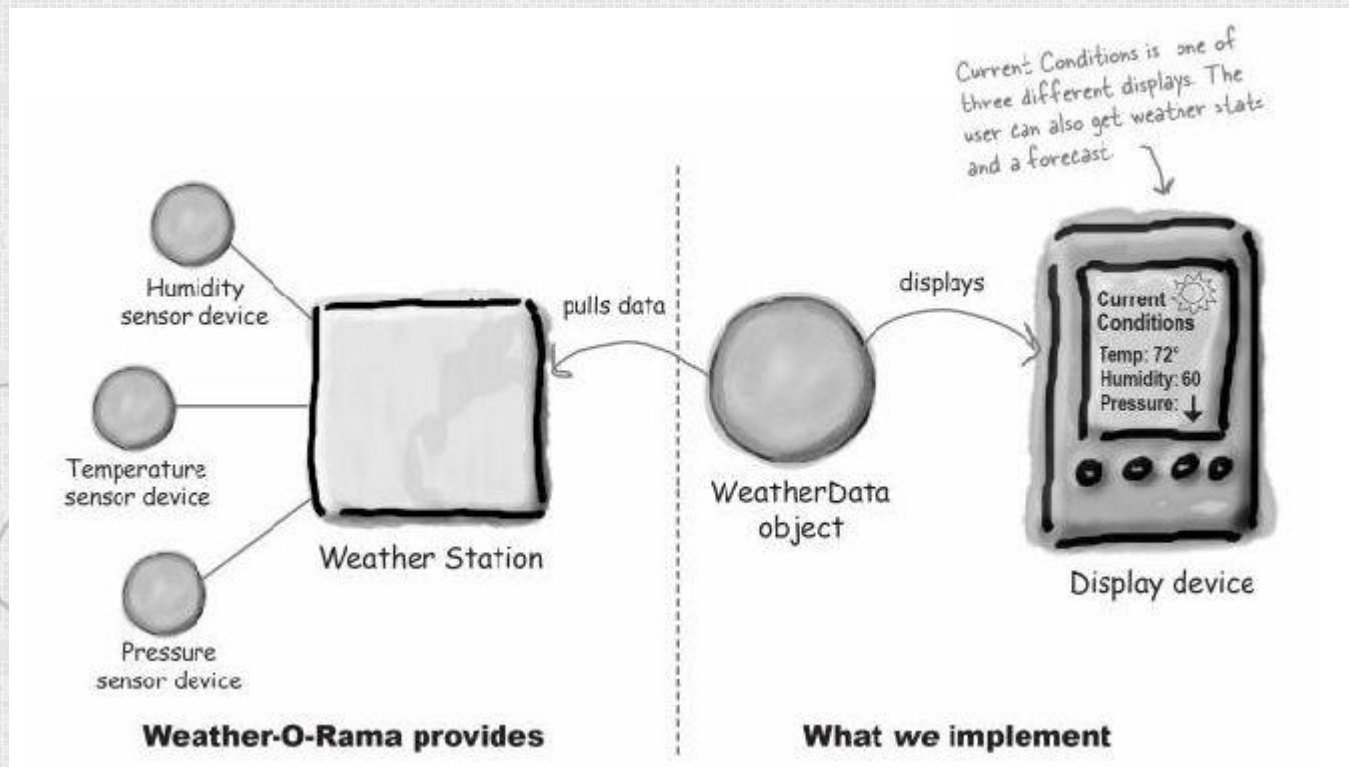


Observer Example



A Problem

- Multiple displays need to be updated with weather data from a single weather station



Display Two



Display Three

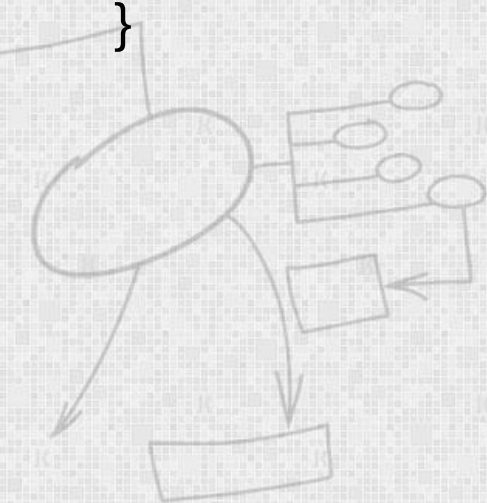


Future displays

Implementing to Interfaces

```
public interface IWeatherPublisher
{
    void AddSubscriber( IWeatherObserver subscriber );
    void RemoveSubscriber( IWeatherObserver subscriber );
}
```

```
public interface IWeatherObserver
{
    void Update( float temp, float humidity, float pressure );
}
```

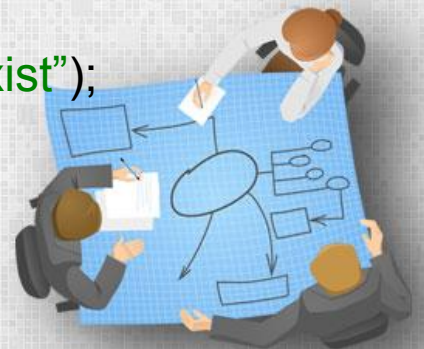


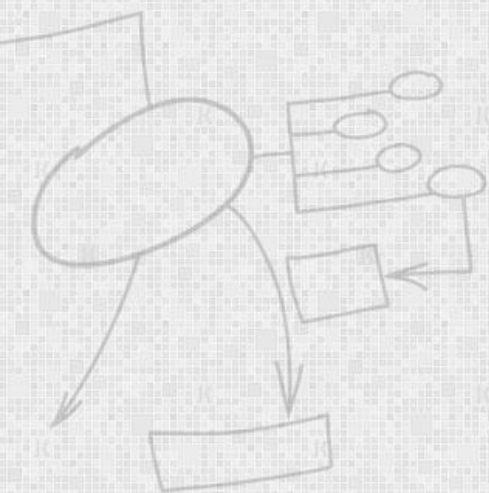
Implementing IWeatherPublisher

```
public class WeatherData : IWeatherPublisher
{
    private IList<IWeatherObserver> observers = new List<IWeatherObserver>();
    private float temperature;
    private float humidity;
    private float pressure;

    public void AddSubscriber( IWeatherObserver subscriber ) {
        observers.Add( subscriber );
    }

    public void RemoveSubscriber( IWeatherObserver subscriber ) {
        if( !observers.Contains(subscriber) )
            throw new ArgumentException("Subscriber does not exist");
        observers.Remove(subscriber);
    }
}
```





Implementing IWeatherPublisher

```
public void SetMeasurements( float temperature, float humidity,  
    float pressure) {  
    this.temperature = temperature;  
    this.humidity = humidity;  
    this.pressure = pressure;  
    notifyObservers();  
}  
private void notifyObservers() {  
    foreach (IWeatherObserver observer in observers)  
    {  
        observer.Update( temperature, humidity, pressure );  
    }  
}  
// other WeatherData methods here - getters
```



Implementing Observers

```
public class CurrentConditionsDisplay : IWeatherObserver {  
    private float temperature;  
    private float humidity;  
  
    public CurrentConditionsDisplay( IWeatherPublisher weatherData ) {  
        weatherData.AddSubscriber( this );  
    }  
  
    public void Update( float temperature, float humidity, float pressure ) {  
        this.temperature = temperature;  
        this.humidity = humidity;  
        Display();  
    }  
  
    public void Display() {  
        System.Console.WriteLine( "Current conditions: " + temperature  
            + "F degrees and " + humidity + "% humidity");  
    }  
}
```



Implementing Observers

```
namespace ObserverDrWasi
{
    class Program
    {
        static void Main(string[] args)
        {
            WeatherData weatherPublisher = new WeatherData();
            CurrentConditionsDisplay Disp1 = new CurrentConditionsDisplay(weatherPublisher);
            CurrentConditionsDisplay Disp2 = new CurrentConditionsDisplay(weatherPublisher);
            CurrentConditionsDisplay Disp3 = new CurrentConditionsDisplay(weatherPublisher);
            weatherPublisher.SetMeasurements(98, 100, 102);
            Disp1.Display();
            Disp2.Display();
            Disp3.Display();

            weatherPublisher.SetMeasurements(100, 99, 103);
            Disp1.Display();
            Disp2.Display();
            Disp3.Display();
        }
    }
}
```



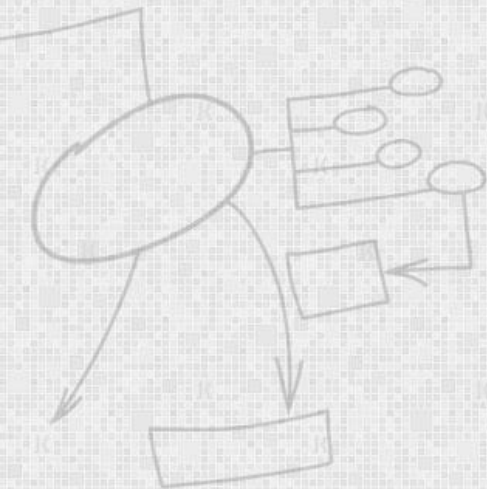
Memento

- [Memento](#) is a [2000 film](#) about Leonard Shelby and his quest to revenge the brutal murder of his wife. Though Leonard is hampered with short-term memory loss, he uses notes and tatoos to compile the information into a suspect.



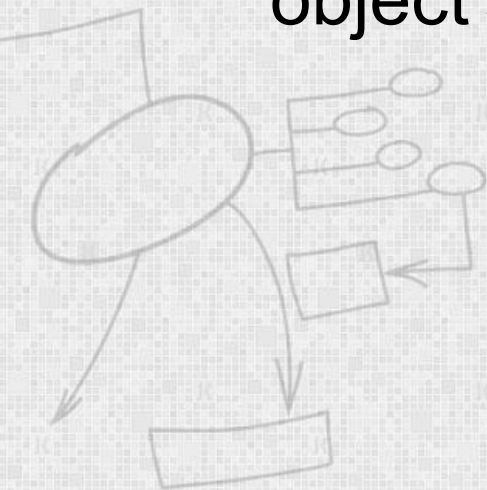
Definition

- The memento captures and externalizes an object's internal state, so the object can be restored to that state later.



Applicability

- Use the Memento pattern when:
 - A snapshot of an object's state must be saved so that it can be restore to that state later, and
 - A direct interface to obtaining the state would expose implementation details and break the object's encapsulation



Participants

- There are 3 participants in the Memento pattern:
 - **Originator:** whose state has to be maintained
 - **Memento:** stores the state of the Originator object
 - **Caretaker:** has a handle to the Memento



```
public class Originator
{
    private String state;

    public void setState(String state)
    {
        this.state = state;
    }

    public String getState()
    {
        return state;
    }

    public Memento saveStateToMemento()
    {
        return new Memento(state);
    }

    public void getStateFromMemento(Memento memento)
    {
        state = memento.getState();
    }
}
```

```
public class Memento
{
    private String state;

    public Memento(String state)
    {
        this.state = state;
    }

    public String getState()
    {
        return state;
    }
}
```

Example

```
public class CareTaker
{
    private List<Memento> mementoList = new List<Memento>();

    public void add(Memento state)
    {
        mementoList.Add(state);
    }

    public Memento get(int index)
    {
        return mementoList[index];
    }
}
```



```
class Program
{
    static void Main(string[] args)
    {
        Originator originator = new Originator();
        CareTaker careTaker = new CareTaker();

        originator.setState("State #1");
        originator.setState("State #2");
        careTaker.add(originator.saveStateToMemento());

        originator.setState("State #3");
        careTaker.add(originator.saveStateToMemento());

        originator.setState("State #4");
        Console.WriteLine("Current State: " + originator.getState());

        originator.getStateFromMemento(careTaker.get(0));
        Console.WriteLine("First saved State: " + originator.getState());
        originator.getStateFromMemento(careTaker.get(1));
        Console.WriteLine("Second saved State: " + originator.getState());
    }
}
```

Example

```
Current State: State #4  
First saved State: State #2  
Second saved State: State #3  
Press any key to continue . . .
```

